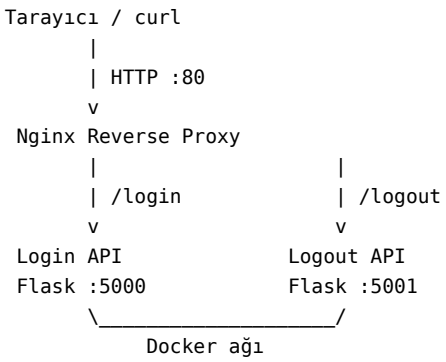
Nginx API Gateway Uygulaması

Ubuntu • Python Flask • Nginx • Docker Compose
Temel ağ bilgisine sahip olanlar için adım adım uygulama rehberi

Bu rehberde iki bağımsız Python API geliştirilecek, Nginx üzerinden localhost:80/login ve localhost:80/logout adreslerine yönlendirilecek ve bütün bileşenler Docker Compose ile çalıştırılacaktır.

1. Ödevin amacı ve mimari

Adres	Beklenen yanıt
http://localhost/login	HTML giriş sayfası
http://localhost/logout	Çıkış yapılmıştır.



Kullanıcı doğrudan Python servislerine değil Nginx'in 80 portuna bağlanır. Nginx isteğin URL yolunu inceleyerek ilgili backend servisine iletir. HTTP için 80 varsayılan port olduğundan tarayıcıda :80 yazmak zorunlu değildir.

2. Ubuntu ortamını hazırlama

Sistemi güncelle

```
sudo apt update
sudo apt upgrade -y
```

Docker ve Compose sürümlerini kontrol et

```
docker --version
docker compose version
```

Docker kurulu değilse resmi Ubuntu rehberini izle: https://docs.docker.com/engine/install/ubuntu/ . Kurulum sonrasında test et:

```
sudo docker run hello-world
```

Sonraki örneklerde kullanıcının Docker çalıştırma yetkisine sahip olduğu varsayılır. Yetki hatası alırsan Docker komutlarını sudo ile çalıştırabilirsin.

3. Proje klasörlerini oluşturma

```
mkdir nginx-api-gateway
cd nginx-api-gateway
mkdir login-api logout-api nginx

nginx-api-gateway/
├── login-api/
│   ├── app.py
│   ├── requirements.txt
│   └── Dockerfile
├── logout-api/
│   ├── app.py
│   ├── requirements.txt
│   └── Dockerfile
├── nginx/
│   └── default.conf
└── docker-compose.yml
```

4. Birinci API: Login sayfası

Flask, Python ile HTTP isteği alan web uygulamaları oluşturmak için kullanılan hafif bir framework'tür. İlk servis GET /login isteğinde HTML sayfası döndürecek.

login-api/app.py

```
from flask import Flask, render_template_string

app = Flask(__name__)

@app.route("/login", methods=["GET"])
def login():
    return render_template_string("""
    <!DOCTYPE html>
    <html lang="tr">
    <head>
        <meta charset="UTF-8">
        <title>Login Sayfası</title>
    </head>
    <body>
        <h1>Giriş Sayfası</h1>
        <form>
            <input type="text" placeholder="Kullanıcı Adı">
            <br><br>
            <input type="password" placeholder="Şifre">
            <br><br>
            <button type="button">Giriş Yap</button>
        </form>
    </body>
    </html>
    """)

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

@app.route URL yolunu tanımlar. render_template_string HTML yanıt üretir. 0.0.0.0, uygulamanın container ağ arayüzlerinden bağlantı kabul etmesini sağlar. 5000, login API'nin dinlediği porttur.

login-api/requirements.txt

```
Flask==3.1.1
```

login-api/Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

Dockerfile image'ın nasıl oluşturulacağını tarif eder. FROM temel image, WORKDIR çalışma dizini, COPY dosya kopyalama, RUN kurulum, CMD başlangıç komutudur. EXPOSE portu belgeler; host üzerinde port yayımlamaz.

5. İkinci API: Logout mesajı

İkinci Flask servisi GET /logout isteğinde tam olarak istenen metni döndürecek.

logout-api/app.py

```
from flask import Flask

app = Flask(__name__)

@app.route("/logout", methods=["GET"])
def logout():
    return "Çıkış yapılmıştır."

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5001)
```

logout-api/requirements.txt

```
Flask==3.1.1
```

logout-api/Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 5001
CMD ["python", "app.py"]
```

Bu iki servis birbirinden bağımsız container'larda çalışır. Login API 5000, logout API 5001 portunu dinler.

6. Nginx Reverse Proxy yapılandırması

nginx/default.conf

```
server {
    listen 80;
    server_name localhost;

    location = /login {
        proxy_pass http://login-api:5000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }

    location = /logout {
        proxy_pass http://logout-api:5001;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
```

listen 80: Nginx 80 portunu dinler. location = /login: yalnızca tam /login yolunu eşleştirir. proxy_pass: isteği backend servisine aktarır. proxy_set_header: Host ve istemci IP bilgilerini backend'e iletmeye yardımcı olur.

Nginx, login-api ve logout-api adlarını Docker'ın dahili DNS servisi sayesinde çözebilir. Bu adlar Docker Compose servis adlarıdır.

7. Docker Compose ile tüm bileşenleri birleştirme

Proje kökündeki docker-compose.yml

```
services:
  login-api:
    build: ./login-api
    expose:
      - "5000"
    networks:
      - api-network

  logout-api:
    build: ./logout-api
    expose:
      - "5001"
    networks:
      - api-network

  nginx:
    image: nginx:stable
    ports:
      - "80:80"
    volumes:
      - ./nginx/default.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on:
      - login-api
      - logout-api
    networks:
      - api-network

networks:
  api-network:
    driver: bridge
```

Anahtar	Görevi
services	Container servislerini tanımlar
build	Dockerfile ile image oluşturur
image	Hazır image kullanır
ports	Host portunu container portuna eşler
expose	Servis portunu belgeler; host’a açmaz
volumes	Dosya/dizin bağlar
depends_on	Başlatma sırasını düzenler
networks	Container iletişim ağını tanımlar

ports: "80:80" host makinenin 80 portunu Nginx container'ının 80 portuna yönlendirir. expose ise host portu açmaz; aynı ağdaki servisler zaten birbirlerine erişebilir. depends_on servislerin hazır olduğuna değil başlatma sırasına ilişkin garanti verir.

8. Projeyi çalıştırma

```
pwd
docker compose config
docker compose up -d --build
docker compose ps
```

config yapılandırmayı doğrular; up -d --build image'ları oluşturup container'ları arka planda başlatır; ps çalışan servislerin durumunu gösterir. Üç servisin de Up durumda olması beklenir.

9. API testleri

Tarayıcı üzerinden

http://localhost/login adresi bir giriş sayfası göstermelidir. http://localhost/logout adresi ise Çıkış yapılmıştır. metnini göstermelidir.

Terminal üzerinden

```
curl -i http://localhost/login
curl -i http://localhost/logout
```

Her iki istekte de normal koşullarda HTTP/1.1 200 OK yanıtı beklenir. Login yanıtının gövdesi HTML, logout yanıtının gövdesi istenen metindir. Bu örnek giriş formu yalnızca görseldir; kullanıcı doğrulama veya gerçek oturum kapatma yapmaz.

10. Ağ seviyesinde istek akışı

```
1. Tarayıcı: GET /login HTTP/1.1
2. Host port yönlendirmesi: localhost:80 -> Nginx:80
3. Nginx: /login kuralını eşleştirir
4. Docker DNS: login-api -> container IP adresi
5. Nginx -> Flask: TCP üzerinden HTTP, port 5000
6. Flask: HTML response oluşturur
7. Flask -> Nginx -> Tarayıcı: HTTP 200 OK
```

DNS servis adını IP'ye çevirir. TCP istemci-Nginx ve Nginx-Flask arasında bağlantı sağlar. HTTP istek/yanıt biçimini belirler. Port ilgili süreci belirler. Docker bridge network container'ların sanal ağda haberleşmesini sağlar. Reverse Proxy istekleri ilgili backend'e iletir.

11. Logları inceleme

```
docker compose logs
docker compose logs nginx
docker compose logs login-api
docker compose logs -f nginx
```

Nginx loglarında /login veya /logout için GET isteğini ve 200 durum kodunu gözlemleyebilirsin. -f canlı log takibi sağlar.

12. Yaygın hatalar ve çözümler

Hata	Olası neden / çözüm
port is already allocated	80 portu kullanımda: sudo ss -ltnp '(sport = :80)'
502 Bad Gateway	Backend erişilemiyor: docker compose logs
404 Not Found	URL yolunu ve Nginx location kuralını kontrol et
Connection refused	docker compose ps ile servis durumunu kontrol et
ModuleNotFoundError	docker compose up -d --build ile image yenile

Host Ubuntu üzerinde ayrı bir Nginx 80 portunu kullanıyorsa ve ihtiyacın yoksa sudo systemctl stop nginx komutuyla geçici olarak durdurabilirsin.

13. Durdurma ve yeniden başlatma

```
docker compose down
docker compose up -d
# Kod veya Dockerfile değiştiyse:
docker compose up -d --build
```

14. Teslim öncesi kontrol listesi

• İki bağımsız Flask API hazır.

• Her API için requirements.txt ve Dockerfile hazır.

• Nginx URL yoluna göre iki backend’e yönlendiriyor.

• Docker Compose üç servisi tek komutla başlatıyor.

• localhost/login giriş sayfası gösteriyor.

• localhost/logout istenen mesajı döndürüyor.

• Backend portları host üzerinde yayımlanmıyor.

Sonuç: Bu çalışma, containerization, reverse proxy, service discovery, port mapping, HTTP routing ve Docker networking kavramlarını uygulamalı olarak gösteren temel bir API Gateway örneğidir. Üretim sistemlerinde ek olarak TLS, kimlik doğrulama, rate limiting, merkezi loglama ve health check kullanılabilir.